ASSIGNMENT 2

Design of a Vector Embedding for Capability Composition

Technical Report

Course: PCCST503

Shima Shamsudheen

Roll No: 77

Department of Computer Science and Engineering

Government Engineering College Thrissur

October 2026

Abstract

This report presents a problem-specific vector embedding for formally specified states, goals and executable capabilities. States and goals are embedded in a signed **condition space** (+1 true, -1 false, 0 unknown). Capabilities are embedded as the concatenation of six weighted blocks: preconditions, effects, inputs, outputs, execution mechanism and resources. Cost, reliability, availability and risk are kept in a separate **operational vector**. Functional resemblance is measured with cosine similarity, while composability is decided by a separate compatibility test on the same blocks, so the two notions are never conflated. Composition is defined by a block algebra that produces a vector for C2 o C1 and, recursively, for C1 o … o Cn. The design was implemented in Python and evaluated on an e-commerce ordering problem (7 capabilities) and a file-processing pipeline (4 capabilities). The five required experiments show that the representation distinguishes compatible from incompatible capabilities, builds composites whose vectors are consistent with their components, separates alternative implementations without treating them as identical, ranks capabilities by goal relevance, and keeps operational attributes separate while still letting them influence composition. All 16 hand-labelled compatibility relations across the two problems were reproduced.

1. Problem Definition

A formally specified application is modelled as A = (S, C, SI, G, R, K): a state space, a set of capabilities, an initial state, a goal specification, resources and global constraints. A capability is a reusable operation described by what it requires (inputs, preconditions, resources), what it does (effects), what it produces (outputs) and the constraints and quality attributes under which it runs. Assignment 1 searched for a sequence of transitions from the initial state to a goal state. This assignment instead asks how the operations themselves should be **represented**.

The central question is: how can formally specified states, goals and executable capabilities be represented in a vector space so that the representation preserves the relationships needed for capability compatibility, composition and the construction of complex application functionality? Unlike Word2Vec, which captures semantic relationships between words, the relationships of interest here are functional: which capabilities can work together, what they require and produce, and how they transform the system state. Planning and replanning algorithms (BFS, A*, LPA*, D* Lite) are outside the scope of this work.

2. Design Requirements

The assignment lists eight properties the representation must investigate. They are restated below as concrete requirements, together with the design decision taken for each.

#	Requirement	Design decision
R1	Capability identity: different capabilities are distinguishable	Separate blocks for P, E, I, O, T, R so any functional difference changes the vector
R2	State awareness: relate capabilities to states	States live in the same condition space as the P and E blocks; applicability is a vector comparison
R3	Precondition-effect compatibility (Ei implies Pj)	Compatibility test on E1 and P2 with signed values, so negation is native
R4	Input-output compatibility	Multi-hot I and O blocks over a shared data vocabulary
R5	Similarity differs from composability	Cosine similarity for resemblance; a separate compatibility test for composition
R6	Composition of two or n capabilities	Block algebra giving a vector for C2 o C1, applied recursively

#	Requirement	Design decision
R7	Goal relevance	Goal embedded in condition space; relevance measures progress on unmet goal literals
R8	Operational properties	Cost, reliability, availability, risk in a separate operational vector; availability also gates compatibility; resources are a block; constraints are checked against the state

3. Related Embedding Approaches

Distributed word representations. Word2Vec (Mikolov et al., 2013) learns dense vectors from co-occurrence so that semantically similar words are close and some relations appear as vector offsets. This works because meaning is reflected in usage statistics. Capability composition is not a statistical-similarity problem: CreateOrder and MakePayment are functionally related but would not necessarily appear close in a learned semantic space, and the validity of a composition is a logical property that distance alone cannot guarantee. A learned model would also need a large corpus of capability descriptions, which a formal specification does not provide.

Knowledge-graph embeddings. Models such as TransE (Bordes et al., 2013) represent relations as translations between entity vectors (h + r is approximately t). A capability graph could be embedded this way, with edges meaning "can follow". However, these models learn from observed edges and cannot determine whether an unseen pair is logically valid, and they do not give a principled way to build a composite vector.

Symbolic action models. STRIPS (Fikes and Nilsson, 1971) and PDDL describe actions by preconditions and effects, which gives exact applicability and composition semantics but no vector representation, so notions of similarity, goal relevance and alternatives are not available directly.

Semantic service matchmaking. Work on semantic web service capability matching (for example Paolucci et al., 2002) compares inputs, outputs, preconditions and effects (IOPE) of services using ontology-based degrees of match. This is the closest conceptual precedent. It is symbolic and does not produce a numeric embedding.

Design choice. The proposed representation combines the strengths of these approaches: the exactness of symbolic IOPE and STRIPS-style semantics, encoded as structured vectors so that cosine similarity, algebraic composition and goal comparison all operate in one space. It is hand-designed rather than learned, which gives guaranteed properties at the cost of adaptivity (see Section 11).

4. Proposed Representation

4.1 Condition space for states and goals

Each Boolean state variable (for example Order.exists) gets one dimension. A state or goal is a vector over these dimensions with value +1 if the variable is true, -1 if false and 0 if unspecified. Because true and false are opposite signs on the same dimension, negation needs no extra machinery: "Order.exists = false" is the exact opposite of "Order.exists = true", which is what separates CancelCart from MakePayment. In the first problem the condition space has 11 dimensions. Numeric and categorical variables (for example Cart.item_count) are handled through constraints rather than vector dimensions.

4.2 Capability space

A capability is embedded as the concatenation of six weighted blocks:

Block	Content	Encoding	Weight
P	Preconditions	Signed (+1/-1/0) over state variables	1.0
E	Effects	Signed (+1/-1/0) over state variables	1.0
I	Required inputs	Multi-hot over data names	0.5
O	Produced outputs	Multi-hot over data names	0.5
T	Type / mechanism	Multi-hot over 9 mechanism types	0.3
R	Resources	Multi-hot over resource names	0.3

Semantic content (P and E) has the largest weight because it defines function. Data (I, O) is weighted at half. The mechanism and resource blocks have a small weight so that capabilities with the same function but different implementations (API, database, GUI) stay close but are not identical. For the first problem the capability space has d = 2(11) + 2(8) + 9 + 6 = 53 dimensions.

4.3 Operational vector (kept separate)

Cost, reliability, availability and risk are placed in a separate four-dimensional vector [cost/10, reliability, availability, 1 - risk]. They are deliberately **not** concatenated into the semantic vector. In an earlier version of the implementation, raw cost values (1 to 5) sat beside 0/1 features and dominated the cosine norm, so expensive capabilities scored lower goal similarity simply for being expensive. Separating the spaces removes this distortion. Availability still has a decisive role: an unavailable capability is excluded by the compatibility test and by applicability. Constraints (for example payment_amount <= transaction_limit) are evaluated against the state rather than embedded.

5. Mathematical Formulation

5.1 Embedding functions

Let V = {x1, ..., xn} be the Boolean state variables, D = {d1, ..., dm} the data names, M the 9 mechanism types and Res the resource names.

$$\varphi_S(S) \in \{-1,\ 0,\ +1\}^n, \quad \varphi_S(S)_k = +1 \text{ if } S(x_k) \text{ is true; } -1 \text{ if false; } 0 \text{ if unspecified}$$

$$\varphi_G(G) \in \{-1,\ 0,\ +1\}^n \quad \text{(defined in the same way over the goal literals)}$$

$$\varphi_C(C) = [\ w_P{\cdot}p\ ;\ w_E{\cdot}e\ ;\ w_I{\cdot}i\ ;\ w_O{\cdot}o\ ;\ w_T{\cdot}t\ ;\ w_R{\cdot}r\ ] \in \mathbb{R}^d, \quad d = 2n + 2m + |M| + |Res|$$

$$\varphi_op(C) = [\ cost/10\ ,\ Rel\ ,\ A\ ,\ 1 - risk\ ] \in \mathbb{R}^4$$

5.2 Similarity

Functional resemblance is the cosine similarity of capability vectors:

$$sim(x, y) = (x \cdot y) \ / \ (\|x\|\ \|y\|)$$

5.3 Compatibility

For an ordered pair (C1, C2), define what is known to hold after C1 as k = e1 where e1 is nonzero, otherwise p1 (a condition required by C1 and not changed by it still holds). C1 -> C2 is **compatible** if and only if all four conditions hold:

- **Logic (E1 implies P2):** for every k with p2_k nonzero, known_k = p2_k.
- **Input-output:** for every j with i2_j > 0, o1_j > 0 or i1_j > 0 (the data is produced by C1 or already in scope).

- **Availability:** A1 = A2 = 1.
- **Non-redundancy:** C2 establishes at least one effect not already true (so a capability followed by itself is not a useful composition).

A graded logic score is also reported, the mean over required conditions of known_k * p2_k, which lies in [-1, 1]: +1 means all required conditions are satisfied, 0 means unknown or mixed, and -1 means they all conflict.

5.4 Goal relevance and applicability

Let s = phi_S(S) and g = phi_G(G). The unmet part of the goal is g* = g where g differs from s, and 0 elsewhere. The relevance of a capability with effect vector e is:

$$rel(C) = (e \cdot g^*) \,/\, \|g^*\|_1 \quad \in [-1, 1]$$

A value of 1 means the capability achieves every unmet goal literal, 0 means it is irrelevant, and a negative value means it works against the goal. A capability is **applicable** in state S if the state satisfies P, all constraints K hold in S, all inputs are available and A = 1.

6. Capability Composition Model

For compatible C1 and C2, the composite C2 o C1 (execute C1, then C2) is defined block-wise:

Component	Rule	Reason
Preconditions P	P = P1	All of P2 is already satisfied by E1 or P1
Effects E	E = E2 where nonzero, otherwise E1	Later effects override earlier ones
Inputs I	I = I1 plus (I2 minus O1)	Only data not produced internally must come from outside
Outputs O	O = O1 union O2	All produced data remains available
Mechanism T, Resources R	Union	The composite needs everything its parts need
Cost	c1 + c2	Additive
Reliability	Rel1 x Rel2	Sequential steps must all succeed
Availability	A1 x A2	Any unavailable step blocks the whole
Risk	1 - (1 - r1)(1 - r2)	Probability that at least one step is risky
Constraints K	Union	All constraints must hold

Composition is applied recursively: compose(C1, ..., Cn) folds the pairwise rule left to right, checking compatibility of each next capability against the composite built so far. The same rule is implemented directly on vectors (compose_vectors) using element-wise operations, and the implementation verifies that block algebra on the vectors equals the encoding of the composite capability. A **plain vector sum** of the components is not used as the composition operator, because it ignores order and validity and double-counts conditions already satisfied internally.

7. Implementation

The system is a single Python file (main.py) using NumPy, pandas and matplotlib. The central class CapabilityEmbedder builds its vocabularies from a problem (its capabilities, states and goals), so the same code runs unchanged on different applications.

Function	Purpose
encode_state(state)	State to condition-space vector
encode_goal(goal)	Goal to condition-space vector
encode(capability)	Atomic or composite capability to semantic vector
compose(c1, ..., cn)	Composite capability, or None if any step is incompatible
compose_vectors(v1, v2)	Block algebra directly on two semantic vectors
similarity(x, y)	Cosine similarity
compatibility(c1, c2)	Logic, IO, availability and non-redundancy checks and the overall result
is_applicable(state, c, data)	State, constraints, inputs and availability check
goal_relevance(c, goal, state)	Progress on unmet goal literals
operational(c)	Operational vector

A short usage example:

```
emb = CapabilityEmbedder(CAPABILITIES, states=[INITIAL_STATE], goals=[GOAL])
chain = emb.compose(CREATE_ORDER, MAKE_PAYMENT, SEND_NOTIFICATION)
emb.is_compatible(CREATE_ORDER, CANCEL_CART)         # False
emb.goal_relevance(chain, GOAL, INITIAL_STATE)       # 1.0
```

Running python main.py executes all experiments, prints the results, writes compatibility_results.csv, operational_results.csv and experimental_dataset.json, and saves three figures.

8. Experimental Methodology

8.1 Datasets

Problem 1 (e-commerce ordering). Initial state: user authenticated, cart exists with 3 items, no order, payment not started, notification not sent. Goal: Order.exists, Payment.success and Notification.sent all true. Capabilities: CreateOrder (API), MakePayment (API), SendNotification (MESSAGE), CancelCart (FUNCTION), UpdateInventory (DATABASE), and two goal-irrelevant capabilities, GenerateReport (COMPUTATION) and SendMarketingEmail (MESSAGE). SendNotification requires Payment.success, so that CreateOrder -> SendNotification is not compatible, as the assignment specifies. Three alternative implementations of CreateOrder (API, DATABASE, GUI) are used for Experiment 3.

Problem 2 (file processing). UploadFile, ScanFile, StoreFile and DeleteFile, with the goal File.stored. It is used only to test consistency of the representation on a different domain.

Every capability specifies preconditions, effects, inputs, outputs, mechanism, cost, reliability, risk, resources and, where relevant, constraints. The full dataset is saved in experimental_dataset.json.

8.2 Procedure

- **Experiment 1:** compare CreateOrder with MakePayment, CancelCart and SendNotification using similarity and compatibility.
- **Experiment 2:** compose CreateOrder -> MakePayment -> SendNotification, compare the composite vector with its components, verify block algebra, and execute the composite from the initial state.
- **Experiment 3:** compare the API, database and GUI versions of CreateOrder.

- **Experiment 4:** rank all capabilities and composites by goal relevance.
- **Experiment 5:** examine cost, reliability, risk, resources, availability and constraints.
- **Consistency and efficiency:** reproduce 16 hand-labelled compatibility relations across two problems; measure vector size and timing.

9. Results

9.1 Experiment 1: capability compatibility

C1	C2	Cosine similarity	Logic score	IO ok	Compatible
CreateOrder	MakePayment	0.025	1.0	True	True
CreateOrder	CancelCart	0.267	0.0	True	False
CreateOrder	SendNotification	0.000	0.0	False	False

The representation separates the compatible pair from both incompatible pairs. CancelCart conflicts with CreateOrder on Order.exists (logic score 0.0), while SendNotification fails both the precondition (Payment.success unknown) and the input (payment_id is not produced). Notably, CancelCart has the **highest** cosine similarity to CreateOrder (0.267) yet is incompatible, while the compatible MakePayment has a similarity of only 0.025.

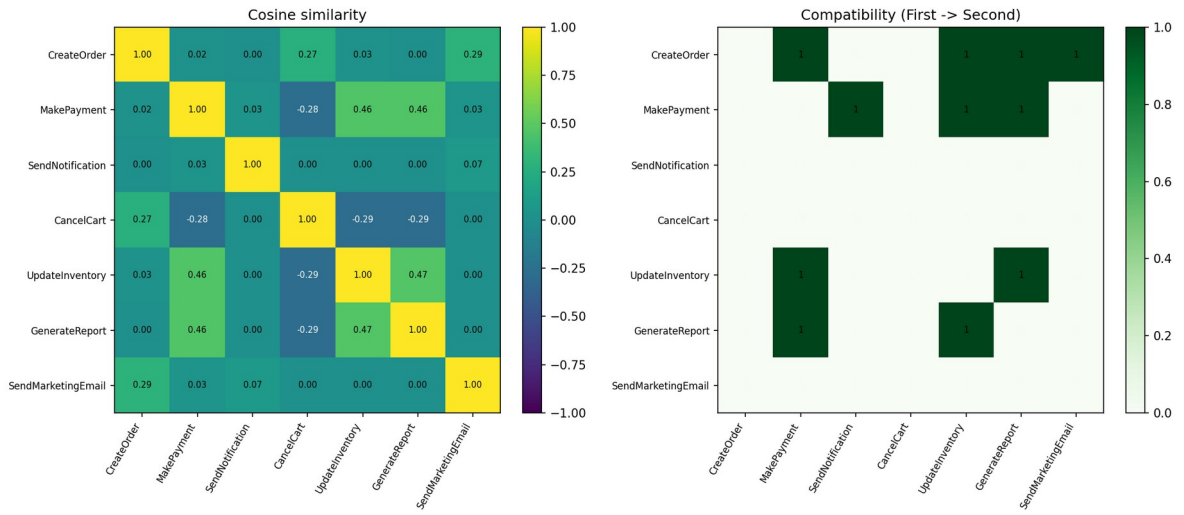


Figure 1: Cosine similarity (left) and compatibility (right) for all ordered pairs.

Across all 42 ordered pairs of distinct capabilities, the correlation between similarity and compatibility is 0.62. The most similar incompatible pairs are SendMarketingEmail -> CreateOrder (0.294) and CreateOrder -> CancelCart (0.267). The least similar compatible pairs include CreateOrder -> GenerateReport (0.000) and CreateOrder -> MakePayment (0.025). Similarity is therefore informative but cannot decide composability, which is why a separate compatibility test is used.

9.2 State awareness

Capability	Applicable in initial state	Applicable after CreateOrder
CreateOrder	True	True
MakePayment	False	True
SendNotification	False	False

Capability	Applicable in initial state	Applicable after CreateOrder
CancelCart	True	False
UpdateInventory	False	True
GenerateReport	False	True
SendMarketingEmail	True	True

Applicability changes with the state in the expected way: MakePayment becomes applicable and CancelCart stops being applicable once an order exists.

9.3 Experiment 2: capability composition

CreateOrder -> MakePayment has cost 7 and reliability 0.9405. The invalid compositions CreateOrder -> CancelCart and CreateOrder -> SendNotification -> MakePayment return None. The three-step composite is:

Property	Value
Preconditions	Cart.exists, User.authenticated
Effects	Order.exists, Cart.locked, Payment.success, Notification.sent
Inputs / Outputs	cart_id / order_id, payment_id, notification_id
Cost / Reliability / Risk	8 / 0.9217 / 0.2552
Resources	AuthToken, Database, Network, PaymentGateway
Constraints	Cart.item_count > 0; payment_amount <= transaction_limit

The cosine similarity between the composite and its components is 0.795 for CreateOrder, 0.333 for MakePayment and 0.318 for SendNotification. CreateOrder is closest because the composite inherits CreateOrder's preconditions and input, which sit in the heavily weighted P and I blocks. Block algebra on the component vectors reproduces encode(composite) exactly (True). Executing the three capabilities from the initial state satisfies the goal (True), and the composite is applicable from the initial state (True).

9.4 Experiment 3: alternative implementations

	CreateOrder_API	CreateOrder_DB	CreateOrder_GUI
CreateOrder_API	1.000	0.971	0.943
CreateOrder_DB	0.971	1.000	0.952
CreateOrder_GUI	0.943	0.952	1.000

The three implementations are highly similar (0.94 to 0.97) but not identical, because the mechanism and resource blocks differ. By contrast, CreateOrder_API versus MakePayment has similarity 0.025. All three have the same compatibility with MakePayment (True, True, True), reflecting identical function, while their operational profiles differ (cost 2, 3, 4; reliability 0.99, 0.97, 0.95), giving a utility ranking of API (0.470), DB (0.297), GUI (0.209) with utility defined as reliability x (1 - risk) / cost.

9.5 Experiment 4: goal relevance

Capability	Relevance	Cosine (full vector, goal)
CreateOrder -> MakePayment -> SendNotification	1.000	0.631
CreateOrder -> MakePayment	0.667	0.464
CreateOrder	0.333	0.264
MakePayment	0.333	0.347
SendNotification	0.333	0.353
UpdateInventory	0.000	0.000
CancelCart	0.000	0.000
SendMarketingEmail	0.000	0.000
GenerateReport	0.000	0.000

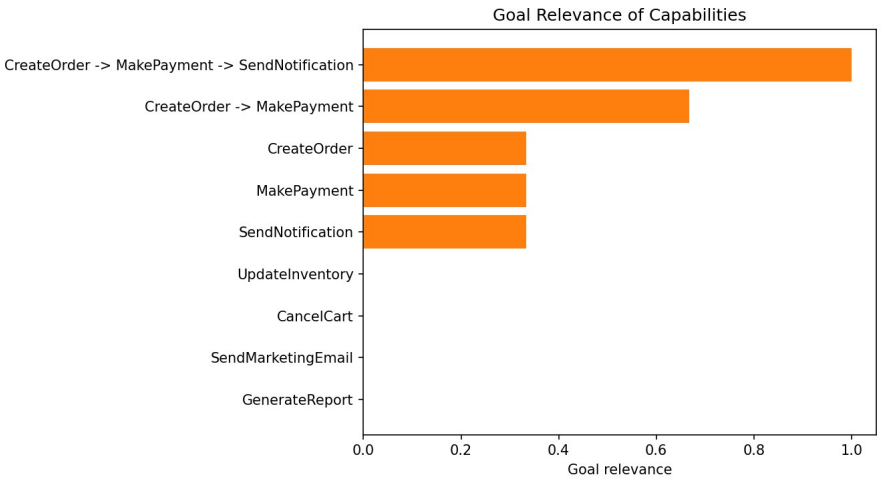


Figure 2: Goal relevance of capabilities and composites.

Each goal-relevant atomic capability achieves one of the three unmet goal literals (relevance 0.333), the two-step composite achieves two (0.667) and the full composite achieves all three (1.0). The four irrelevant capabilities score exactly 0, so the representation separates useful from irrelevant ones and shows progress growing with composition.

9.6 Experiment 5: operational attributes

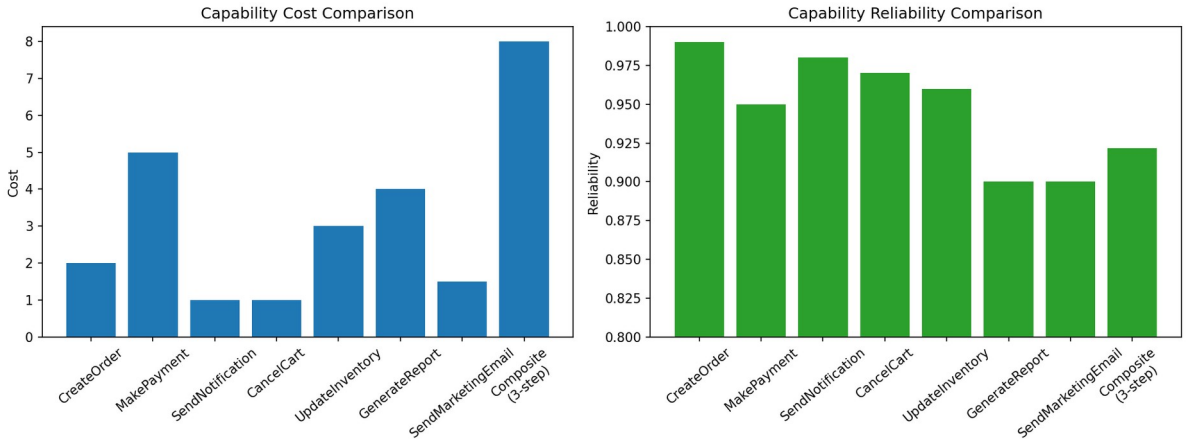


Figure 3: Cost (left) and reliability (right) of the atomic capabilities and the three-step composite.

Costs add (composite 8) and reliabilities multiply (0.9217), so reliability falls as chains lengthen. The composite resource set is the union of its parts.

- **Availability.** Marking MakePayment unavailable leaves its semantic cosine to the original at 1.0, but CreateOrder -> MakePayment changes from compatible (True) to incompatible (False) and the composition returns None. Availability therefore changes composability without distorting semantic similarity.
- **Constraints.** MakePayment is applicable after CreateOrder with payment_amount = 120 (True) but not with payment_amount = 900 against a limit of 500 (False).
- **Risk and resources.** Risk composes as 1 - product of (1 - r), giving 0.2552 for the chain; resources union to four items.

9.7 Consistency and efficiency

On Problem 1, 10 of 10 hand-labelled compatibility relations were reproduced; on Problem 2, 6 of 6, including the composite UploadFile -> ScanFile -> StoreFile (goal relevance 1.0, versus 0.0 for DeleteFile). The same code and parameters were used for both problems. Each capability vector has 53 dimensions (424 bytes in float64). On the development machine, encoding takes about 24 microseconds and a compatibility check about 69 microseconds. Storage is O(N d) for N capabilities and all-pairs compatibility costs O(N^2 (n + m)).

10. Analysis

The table below evaluates the representation against each criterion from the assignment.

Criterion	Question	Evidence
Capability representation	Distinct capabilities distinguishable?	Yes. Maximum off-diagonal cosine is 0.466; no two capabilities coincide.
State relationship	Capability-state relation captured?	Yes. Applicability from a state vector; MakePayment and CancelCart flip after CreateOrder.
Precondition-effect compatibility	Composable vs incompatible?	Yes. Signed values detect conflicts (CancelCart) and unmet conditions (SendNotification).
Input-output compatibility	Dependencies representable?	Yes. I and O blocks; SendNotification needs payment_id from MakePayment.

Criterion	Question	Evidence
Composition	Complex from simple?	Yes. Block algebra equals encode(composite); works for n steps.
Goal relevance	Capability-goal relation identified?	Yes. Relevance 0.333 / 0.667 / 1.0 for 1 / 2 / 3 steps; irrelevant capabilities score 0.
Operational properties	Cost, reliability, availability, constraints?	Yes. Separate operational vector; availability gates compatibility; constraints checked on state.
Consistency	Consistent across problems?	Yes. 16/16 relations on two domains with identical code.
Efficiency	Computational and storage cost?	53 dimensions, 424 bytes per capability, microsecond-scale operations.

Similarity versus composability. The experiments confirm that the two notions differ. CreateOrder and CancelCart share condition dimensions and therefore look similar (0.267), yet they cannot compose because their requirements on Order.exists conflict. CreateOrder and GenerateReport have similarity 0.000 yet compose. Similarity is useful for finding **substitutes** (for example the three CreateOrder implementations) and for ranking candidates, whereas compatibility decides whether a sequence is valid. Keeping both, computed from the same blocks, lets the representation answer both kinds of question.

Design trade-offs. Separating operational attributes from semantics was important: it prevented cost from distorting goal similarity and let availability act as a gate rather than a similarity feature. Weighting the mechanism block low produced the intended behaviour for alternative implementations: close but distinguishable. The composite vector is closest to its first component because preconditions and inputs are inherited from it, which is a faithful consequence of the composition rule rather than a defect. A plain vector sum had a high cosine with the composite (0.864) but is not a valid composition operator because it carries no notion of order or validity.

11. Limitations

- **Hand-designed, not learned.** The encoding and the block weights (1.0, 1.0, 0.5, 0.5, 0.3, 0.3) were chosen by reasoning and testing, not optimised. The cosine values depend on these weights, although the compatibility decisions do not.
- **Boolean conditions only.** Numeric and categorical variables are handled through constraints evaluated on the state, not embedded geometrically, so the vector does not capture, for example, how large a quantity must be.
- **Compatibility is a symbolic rule on the vector blocks.** It is exact and cheap, but it is not a purely geometric property of the embedding (for example not a learned distance threshold).
- **Small datasets.** The evaluation uses 7 + 4 capabilities and two problems. The expected compatibility relations were labelled by hand, so a 16/16 match shows the implementation matches the specification, not that the design would generalise to large, noisy capability catalogues.
- **Static availability.** Availability is a 0/1 flag; time-varying availability A(t) and richer quality attributes (energy, monetary cost) are not modelled.
- **No semantic knowledge across vocabularies.** Two capabilities that use different names for the same condition or data item are treated as unrelated unless the vocabulary is aligned.
- **Sequential composition only.** Parallel and conditional composition are not represented.

12. Conclusion

A structured, problem-specific vector representation can capture the functional relationships required to compose formally specified capabilities. States and goals share a signed condition space with capability preconditions and effects, which makes negation, state awareness, compatibility and goal relevance directly computable. Cosine similarity captures functional resemblance, while a separate compatibility test determines composability, and the experiments show these are genuinely different relations. Composition is a well-defined block algebra that yields a vector for any valid chain, with cost, reliability, risk and resources aggregating correctly. The same implementation reproduced all expected relations on two different problems.

Future work includes learning the block weights or a dense projection from data while keeping the symbolic guarantees, extending the condition space to numeric and categorical variables, modelling time-varying availability, supporting parallel and conditional composition, and connecting the representation to the Assignment 1 planner so that capability embeddings guide search.

References

[1] Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.

[2] Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., and Yakhnenko, O. (2013). Translating embeddings for modeling multi-relational data. Advances in Neural Information Processing Systems 26.

[3] Fikes, R. E., and Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. Artificial Intelligence, 2(3-4), 189-208.

[4] Paolucci, M., Kawamura, T., Payne, T. R., and Sycara, K. (2002). Semantic matching of web services capabilities. Proceedings of the International Semantic Web Conference (ISWC 2002).